

Documentation Technique de l'Application

I. Réflexions initiales sur les choix technologiques

La conception de l'application *Vite & Gourmand* repose sur une architecture découplée (API REST / Front-end), un choix stratégique qui sépare la logique métier de l'interface utilisateur afin de garantir la flexibilité, la sécurité et la maintenabilité du système.

1. Le choix du Back-end : Symfony 7

Le framework **Symfony 7** a été retenu pour propulser l'API REST en raison de sa robustesse industrielle et de sa rigueur structurelle. Il intègre nativement des composants de sécurité indispensables pour une application transactionnelle de restauration :

- **Composant Security** : Configuration d'un pare-feu (*firewall*) et gestion fine des droits d'accès selon une hiérarchie de rôles stricte (ROLE_USER, ROLE_EMPLOYEE, ROLE_ADMIN).
- **Password Hasher** : Hachage automatique et sécurisé des mots de passe en base de données à l'aide d'algorithmes cryptographiques récents.
- **Composant Validator** : Vérification systématique de l'intégrité des données reçues par l'API (validation des formats d'e-mail, des numéros de téléphone et des champs obligatoires).
- **Doctrine ORM** : Abstraction complète de la base de données permettant de manipuler les entités sous forme de composants PHP et d'assurer des requêtes optimisées et sécurisées contre les injections SQL.

Authentification Stateless : JWT (JSON Web Token)

Pour sécuriser les échanges entre le Front-end et l'API sans surcharger le serveur, l'authentification s'appuie sur le protocole **JWT** via le bundle `LexikJWTAuthenticationBundle`. À la connexion, l'API génère un jeton signé cryptographiquement. Ce jeton est stocké côté client et transmis dans l'en-tête HTTP de chaque requête, garantissant une architecture *Stateless* performante et hautement sécurisée.

2. Le choix du Front-end : Bootstrap 5 & Sass

Pour l'interface utilisateur, le framework **Bootstrap 5** a été choisi pour sa rapidité d'intégration et sa gestion native du *Responsive Web Design* (approche mobile-first), essentielle pour les clients passant commande depuis leur smartphone. L'utilisation des préprocesseurs **Sass** permet de surcharger proprement les variables du framework (couleurs, typographies) afin de respecter la charte graphique unique de *Vite & Gourmand*.

3. Base de données et Déploiement

- **MySQL 8** : Choisi pour la persistance des données relationnelles, garantissant l'intégrité et la cohérence des tables complexes (liaisons entre utilisateurs, menus, plats et commandes).
- **Render** : La plateforme Cloud **Render** a été sélectionnée pour l'hébergement du Web Service (API et Front) et du Database Service, offrant une infrastructure de production fiable, moderne et entièrement automatisée via l'utilisation du *GitHub Student Developer Pack*.

II. Configuration de l'environnement de travail

Pour développer sur un système Windows de manière isolée et professionnelle, l'intégralité de l'environnement local a été conteneurisée.

1. Orchestration locale (Docker)

L'application ne dépend d'aucun serveur global installé sur la machine hôte. Elle est orchestrée par **Docker** et **Docker Compose** à travers quatre services cloisonnés et interconnectés :

- **vite_gourmand_front** : Serveur Apache (httpd) hébergeant l'interface utilisateur (HTML5, Bootstrap 5, Sass) sur le port 8080.
- **vite_gourmand_back** : Environnement d'exécution PHP 8.2 exécutant le framework Symfony 7 et l'API REST sur le port 8000.
- **symfony_db** : Serveur de base de données relationnelle **MySQL 8** (port interne 3306).
- **symfony_pma** : Interface graphique **PhpMyAdmin** accessible sur le port 8081 pour l'administration visuelle et l'importation du fichier database.sql.

2. Outils et Workflow de développement

- **Éditeur de code** : Visual Studio Code, enrichi des extensions spécialisées PHP (IntelliSense, PHP Namespace Resolver) et Git (GitLens) pour optimiser la productivité.
 - **Gestion de versions (Git)** : Utilisation rigoureuse de dépôts sur **GitHub** structurés autour de deux branches permanentes : main (dédiée aux versions stables de production) et develop (branche de centralisation des fonctionnalités), complétées par des branches éphémères (feat/...) pour chaque module développé.
 - **Gestionnaires de dépendances** : **Composer** pour l'écosystème de paquets PHP du Back-end et **npm** pour la compilation des assets du Front-end.
-
- **Tests d'API** : Utilisation de **Postman** pour valider la conformité des routes de l'API REST, tester les structures de réponses JSON et simuler l'envoi des jetons JWT dans les en-têtes de requêtes.

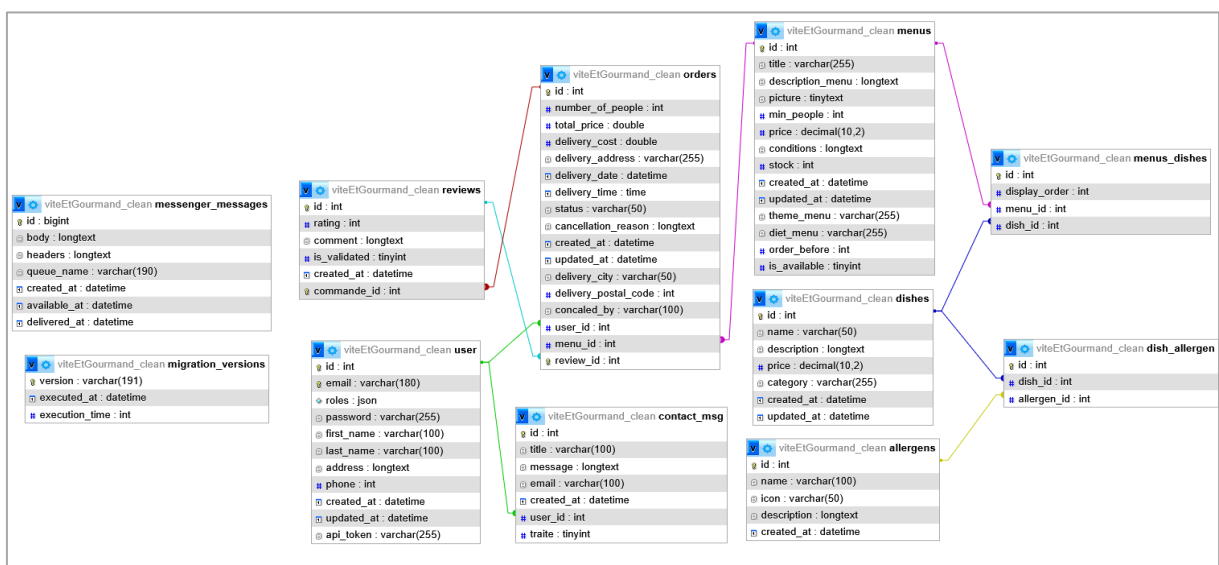
III. Modèle Conceptuel de Données (MCD)

Le schéma relationnel de la base de données a été modélisé de manière à refléter scrupuleusement les règles métiers du service de traiteur. Chaque entité est structurée avec ses propriétés et ses relations associées

1. Diagramme de classe

Le diagramme de classes généré via **phpMyAdmin** illustre la structure objet et les relations entre les entités métier et il met en évidence :

- ✓ les relations OneToMany et les relations ManyToMany

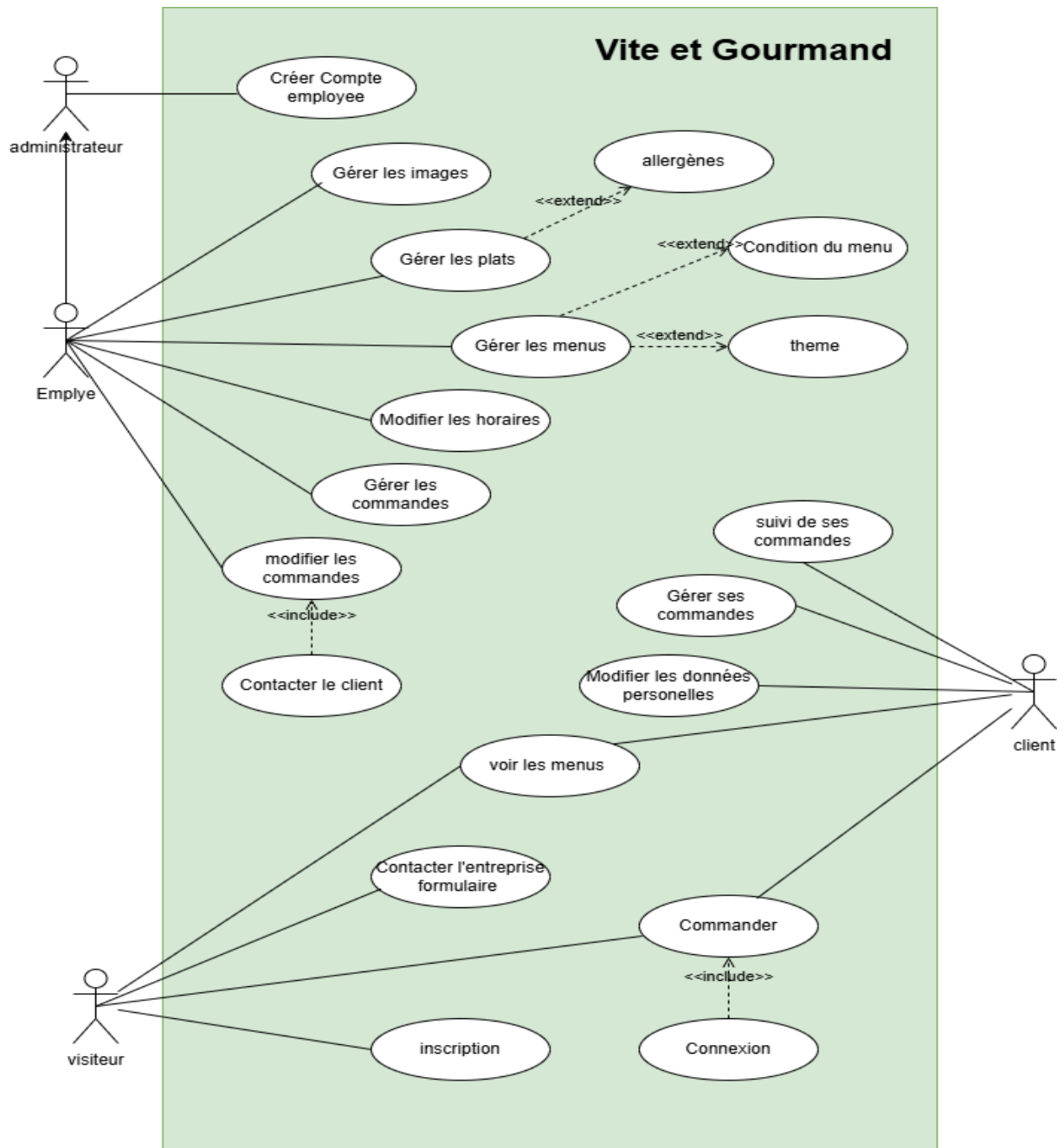


Une capture complète de la base de données générée depuis **phpMyAdmin** est fournie.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ allergens	★ Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_0900_ai_ci	16.0 Kib	-
☐ contact_msg	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_0900_ai_ci	32.0 Kib	-
☐ dishes	★ Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_0900_ai_ci	16.0 Kib	-
☐ dish_allergen	★ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_0900_ai_ci	48.0 Kib	-
☐ menus	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_0900_ai_ci	16.0 Kib	-
☐ menus_dishes	★ Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_0900_ai_ci	48.0 Kib	-
☐ messenger_messages	★ Browse Structure Search Insert Empty Drop	57	InnoDB	utf8mb4_0900_ai_ci	240.0 Kib	-
☐ migration_versions	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_0900_ai_ci	16.0 Kib	-
☐ orders	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_0900_ai_ci	64.0 Kib	-
☐ reviews	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_0900_ai_ci	32.0 Kib	-
☐ user	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_0900_ai_ci	32.0 Kib	-
11 tables	Sum	109	InnoDB	utf8mb4_0900_ai_ci	560.0 Kib	0 B

1. Diagramme de cas d'utilisation (Use Case)

Le diagramme de cas d'utilisation présente les interactions entre les différents acteurs de l'application :



2. Scripts SQL des tables

```
CREATE TABLE `contact_msg` (  
  `id` int(11) NOT NULL,  
  `title` varchar(100) NOT NULL,  
  `message` longtext NOT NULL,  
  `email` varchar(100) NOT NULL,  
  `created_at` datetime NOT NULL,  
  `user_id` int(11) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
CREATE TABLE `dishes` (  
  `id` int(11) NOT NULL,  
  `name` varchar(50) NOT NULL,  
  `description` longtext NOT NULL,  
  `price` decimal(10,2) NOT NULL,  
  `category` varchar(255) NOT NULL,  
  `created_at` datetime NOT NULL,  
  `updated_at` datetime DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
CREATE TABLE `dish_allergen` (  
  `id` int(11) NOT NULL,  
  `dish_id` int(11) DEFAULT NULL,  
  `allergen_id` int(11) DEFAULT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

```
-- CREATE TABLE `menus_dishes` (  
  `id` int(11) NOT NULL,  
  `display_order` int(11) NOT NULL,  
  `menu_id` int(11) DEFAULT NULL,  
  `dish_id` int(11) NOT NULL  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

CREATE TABLE `menus` (

```
`id` int(11) NOT NULL,  
`title` varchar(255) NOT NULL,  
`description_menu` longtext NOT NULL,  
`picture` longblob DEFAULT NULL,  
`min_people` int(11) NOT NULL,  
`price` decimal(10,2) NOT NULL,  
`conditions` longtext NOT NULL,  
`stock` int(11) NOT NULL,  
`created_at` datetime NOT NULL,  
`updated_at` datetime DEFAULT NULL,  
`theme_menu` varchar(255) DEFAULT NULL,  
`diet_menu` varchar(255) DEFAULT NULL,  
`order_before` int(11) NOT NULL,  
`is_available` tinyint(4) DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

CREATE TABLE `user` (

```
`id` int(11) NOT NULL,  
`email` varchar(180) NOT NULL,  
`roles` longtext CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL CHECK  
(json_valid(`roles`)),
```

```
`password` varchar(255) NOT NULL,  
`first_name` varchar(100) NOT NULL,  
`last_name` varchar(100) NOT NULL,  
`adress` longtext DEFAULT NULL,  
`phone` int(11) DEFAULT NULL,  
`created_at` datetime NOT NULL,  
`updated_at` datetime DEFAULT NULL,  
`api_token` varchar(255) NOT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

CREATE TABLE `orders` (

```
`id` int(11) NOT NULL,  
`number_of_people` int(11) NOT NULL,  
`total_price` double NOT NULL,  
`delivery_cost` decimal(10,0) DEFAULT NULL,  
`delivery_address` varchar(255) NOT NULL,  
`delivery_date` datetime NOT NULL,  
`delivery_time` time NOT NULL,  
`status` varchar(50) NOT NULL,  
`cancellation_reason` longtext DEFAULT NULL,  
`created_at` datetime NOT NULL,  
`updated_at` datetime DEFAULT NULL,  
`delivery_city` varchar(50) NOT NULL,  
`delivery_postal_code` int(11) NOT NULL,  
`concaled_by` varchar(100) DEFAULT NULL,  
`user_id` int(11) NOT NULL,  
`menu_id` int(11) NOT NULL,  
`review_id` int(11) DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

CREATE TABLE `reviews` (

```
`id` int(11) NOT NULL,  
`rating` int(11) NOT NULL,  
`comment` longtext DEFAULT NULL,  
`is_validated` tinyint(4) NOT NULL DEFAULT 0,  
`created_at` datetime NOT NULL,  
`commande_id` int(11) DEFAULT NULL
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```


IV. Documentation du déploiement de l'application sur Render

L'application *Vite & Gourmand* est officiellement déployée en ligne et accessible en production pour les utilisateurs et le jury à l'adresse suivante :

<https://vite-et-gourmand-zhri.onrender.com>.

Le processus de mise en production a été réalisé en connectant directement l'infrastructure Cloud de la plateforme **Render** au dépôt distant **GitHub**, exploitant ainsi les avantages du *GitHub Student Developer Pack*.

Étape 1 : Création et configuration du service de Base de Données

Avant de déployer le code de l'application, l'infrastructure de stockage a été mise en place pour accueillir les données :

Création d'un **Database Service** (serveur de base de données) sur le tableau de bord Render.

Sélection de la région géographique (FRANKFURT) afin de minimiser la latence réseau lors des requêtes.

Récupération des identifiants de connexion sécurisés et de la variable globale DATABASE_URL générée automatiquement par la plateforme pour l'accès aux données.

Étape 2 : Création et liaison du Web Service

Pour propulser l'application Symfony 7 et l'interface utilisateur découplée, un **Web Service** a été initialisé :

Connexion sécurisée du compte GitHub à la plateforme Render.

Sélection du dépôt officiel du projet : Restaurant-vite-et-gourmant.

Liaison de la branche principale (main) comme branche de référence absolue pour l'environnement de production. Tout code fusionné sur cette branche est considéré comme stable et prêt à être publié.

Étape 3 : Injection sécurisée des variables d'environnement

Afin de dissocier strictement le code source des clés de sécurité et des identifiants (conformément aux bonnes pratiques des architectures cloud), les variables de configuration ont été injectées directement via l'onglet **Environnement** du tableau de bord de Render :

APP_ENV=prod : Configure le framework Symfony pour s'exécuter en mode production (activation de l'optimisation des performances et désactivation de la barre de débogage).

APP_SECRET : Clé unique de sécurité indispensable au chiffrement des sessions et des jetons de l'application.

DATABASE_URL : Lien d'accès direct et sécurisé à l'instance de base de données managée.

Étape 4 : Configuration des commandes de Build et de Démarrage

Pour permettre à la plateforme d'installer, de compiler et d'exécuter l'application de manière autonome à chaque mise à jour, les scripts d'automatisation suivants ont été paramétrés :

Commande de construction (Build Command) :

Bash

```
composer install --no-dev --optimize-autoloader && php bin/console cache:clear
```

Cette suite de commandes installe les dépendances PHP de production en excluant les outils de développement, optimise l'autoloader pour accélérer le chargement des classes, et vide le cache système pour intégrer les dernières modifications.

Commande de démarrage (Start Command) : Configure le serveur web de production pour pointer directement vers le dossier public de Symfony (/public/index.php), qui sert de point d'entrée unique et sécurisé pour l'application.

Étape 5 : Pipeline de Déploiement Continu (CI/CD)

Le flux de mise à jour de l'application est désormais entièrement automatisé :

Dès qu'une nouvelle fonctionnalité est validée sur la branche de développement et fusionnée (*merged*) sur la branche main du dépôt GitHub, un *Webhook* transmet instantanément l'alerte à Render.

Render déclenche automatiquement le processus de *Build* en arrière-plan, télécharge le nouveau code et installe les dépendances.

Une fois les étapes de construction validées avec succès, l'ancienne version est remplacée par la nouvelle de manière transparente et sans aucune interruption de service pour les clients du restaurant (*Zero-Downtime Deployment*). En cas d'erreur durant le build, l'ancienne version reste en ligne, garantissant la haute disponibilité du site.

V. Documentation API — Nelmio

Les pages générées par **NelmioApiDocBundle** sont fournies pour documenter les routes API.

Allergens Allergens		^
POST	/api/allergens/new Créer un nouvel allergène	🔒 ↓
GET	/api/allergens/{id} Récupérer un allergène par ID	🔒 ↓
PUT	/api/allergens/{id} Mettre à jour un allergène	🔒 ↓
DELETE	/api/allergens/{id} Supprimer un allergène	🔒 ↓
GET	/api/allergens/ Récupérer la liste de tous les allergènes	🔒 ↓
DishAllergen DishAllergen		^
GET	/api/dish_allergen/{id} Récupérer la liste des allergènes associés à un plat	🔒 ↓
POST	/api/dish_allergen/{id} Ajouter un allergène à un plat	🔒 ↓
DELETE	/api/dish_allergen/{id} Supprimer un allergène d'un plat	🔒 ↓
GET	/api/dish_allergen/{id}/detail Récupérer le détail d'un allergène associé à un plat	🔒 ↓
Dish Dish		^
POST	/api/dishes/new Créer un nouveau plat	🔒 ↓
GET	/api/dishes/list Lister tous les plats	🔒 ↓
GET	/api/dishes/{id} Afficher les détails d'un plat	🔒 ↓
PUT	/api/dishes/{id} Mettre à jour un plat	🔒 ↓
DELETE	/api/dishes/{id} Supprimer un plat	🔒 ↓

Menu Menu		^
POST	/api/menu/new Créer un nouveau menu	🔒 ↓
GET	/api/menu/list Lister les menus avec filtres optionnels	🔒 ↓
GET	/api/menu/{id} Afficher les détails d'un menu	🔒 ↓
PUT	/api/menu/{id} Modifier un menu existant	🔒 ↓
DELETE	/api/menu/{id} Supprimer un menu	🔒 ↓
POST	/api/menu/{id}/picture Uploader une image pour un menu	🔒 ↓
MenuDish MenuDish		^
POST	/api/menus-dishes/{id} Ajouter un plat à un menu	🔒 ↓
GET	/api/menus-dishes/{id}/list Récupérer la liste des plats d'un menu	🔒 ↓
GET	/api/menus-dishes/{id}/detail Récupérer les détails d'un menu avec ses plats et allergènes	🔒 ↓
DELETE	/api/menus-dishes/{menuId}/remove-dish/{dishId} Retirer un plat d'un menu	🔒 ↓

Orders Orders			^
POST	/api/orders/new	Créer une nouvelle commande	🔒
POST	/api/orders/preview	Prévisualiser une commande avant de la créer	🔒
GET	/api/orders/{id}	Afficher les détails d'une commande	🔒
GET	/api/orders	Lister les commandes de l'utilisateur connecté	🔒
PUT	/api/orders/{id}/cancel	Annuler une commande	🔒
PUT	/api/orders/{id}/edit	Modifier une commande	🔒
DELETE	/api/admin/orders/{id}	Supprimer une commande	🔒
GET	/api/admin/orders	Lister toutes les commandes (admin/employee)	🔒
PUT	/api/admin/orders/{id}/status	Mettre à jour le statut d'une commande (admin)	🔒
GET	/api/admin/orders/statistics	Afficher les statistiques de commandes (admin)	🔒
Authentication Authentication			^
POST	/api/register		🔒
POST	/api/login		🔒
GET	/api/user	compte utilisateur	🔒
PUT	/api/user	Edit user profile	🔒
POST	/api/logout	Logout endpoint	🔒

default			^
GET	/api/contact/all		🔒
POST	/api/contact/add	Ajouter un message de contact	🔒
DELETE	/api/contact/{id}	Supprimer un message de contact	🔒
PUT	/api/contact/{id}/treated	Marquer un message de contact comme traité	🔒
GET	/api/debug-db		🔒
PUT	/api/debug-db		🔒
POST	/api/debug-db		🔒
DELETE	/api/debug-db		🔒
OPTIONS	/api/debug-db		🔒
HEAD	/api/debug-db		🔒
PATCH	/api/debug-db		🔒
TRACE	/api/debug-db		🔒
PUT	/api/admin/orders/{id}/edit		🔒
GET	/api/admin/employees		🔒
DELETE	/api/admin/employees/{id}		🔒
DELETE	/api/user/{id}		🔒